Modular Programming: 2-dars

Git commands

+ Buyruqlar (commands) orqali kompyuterni bash-qaradi.
+ Git vs linux (Ubuntu)
+ Git -- uy ichidagi oshxona
+ Ubuntu = - katta restoran
+ Oxshash commandalar

mkdir folder -> papka ochish
cd folder -> papka. ichiga kirish.
rm file -> file ōchirish
touch file -> file ochish

mkdir folder => papka ~~ochirish~~ yaratish (make directory)
rm -r folder => (remove recursive. (ichidagi barcha fayllar b/n ōchirish))

No.

pwd qilsak, bizning turgan joyimizni ko'rsatadi.

1 - step: Desktopga o'tish: cd Desktop

2 - step: Papka yaratish: mkdir git_project

3 - step: yaratgan papkaga kirish: cd git_project va pwd

4 - step: endi fayl ochish uchun esa:
touch preprocessing.ipynb

5 - step: Agar faylni o'chirpchi bolsam:
rm preprocessing.ipynb desam boldi.

6 - step: Endi biz papka o'chiramiz desak:
rm -r Data agar papka bosh bolsa: rmdir folder

Project structure (modular)

1. Desktop yoki kerakli joyga o'ting
2. Papka oching
3. Hidden folder (.git) yarating
   - Update
   - Changes
   - control
4. Vs codedan aynan shu projectni oching
5. Kerakli modular papkani oching

No.

6. O'zgarish/ni saqlang.
7. Githubga ulash

① ② ③
→ Create folder → Git init → Structure →
④ ⑤
→ VS Code → Github link

1-step: git init

2-step: Biz uni Vs codeda ochamiz desak:
code .

3-step: Data ni ichiga 3ta papka ochamiz
mkdir Raw_Data Preprocesed_Data Engineered_Data
→bu yerda vergul yoq va _ bo'lishi kerak sababi uni 3ta alohida papka yaratib qo'yadi.

4-step touch README.md yaratamiz.

5-step: mkdir Notebooks => cd Notebooks =>
Bunda asosan xomaki ishlarni shunday o'tkazish mumkin.
=> touch data_loading.ipynb data_preprocessing.ipynb
data_analysis.ipynb feature_creation.ipynb

↓
ls
↓
touch README.md => Bunga ham o'zgina farq
(papkada nima bor/yo'q
shuni u-n bu papkani
ochgandi)

No.

6 - step: Bitta src ~~file~~ papka ochamiz ichida bizning class filelarimiz boladi.

mkdir src => cd src = ls

7 - step: Class (.py) filelarni yaratish

touch ~~mkdir~~ data_loader.py preprocessed.py tuning.py models.py analysis.py

! Proyektimiz u-n kerak boladigan classlarni tartibli ravishda papkalarga ajratish src folderi ichida boladi.

touch README.md

8 - step: scripts degan folder ochamiz

=> mkdir scripts => cd scripts => ls

touch data_load.py data_preprocessing.py train.py test.py evaluate.py tuning.py README.md

ls (scripts ichida haqiqiy proyektimiz jarayonini kodlar bilan)

9 - step: models folder yaratish ichida bizga .pkl filelarimiz tushadi.

mkdir models

10 - step: results degan folder ochamiz

mkdir results => cd results = ls

mkdir figures tables

cd figures (png rasmlar) ~~tabulate natijalari~~

touch README.md

No.

10-step: log degan papka ochamiz. Bilayotgan o'zgarish/imizni alohida papkani ichiga saqlab ketamiz. mkdir log => cd log => ls
(dastur ishlaganda yozib boriladigan journal (log) fayllar.
↳ touch data_loader.log preprocessor.log feature_selection.log trainer.log evaluator.log
↳ ls
↳ Bamisoli CCTV (black box).

11-step: errors papka ochamiz
mkdir errors => cd errors => ls
↳ touch training_errors.log testing_errors.log

12-step: test degan folder ochamiz
(offline testing; alohida bitta boshida 80%/20% yoki 70/30 qilib bilib olayapmizku, unaga emas - alohida output qiymati yo'q datasetni beramiz
mkdir test => cd test => ls

13-step: pipeline degan folder ochamiz.
(jarayonni avtomatlashtirish uchun kerak bo'ladigan papka)
mkdir pipeline => cd pipeline => ls

14-step: requirements.txt
↳ touch requirements.txt

15-step: ~~mkdir~~ touch README.md

No.

16-step: touch .gitignore
Gitga qaysi fayl yoki papkalarni kuzatmaslik (ignore qilish) kerakligini aytadigan fayl

Masalan: logs/

- __pycache__/ → Python yaratgan ~~a~~ adigan vaqtinchalik fayllarni ignore qiladi
- *.pyc → .pyc bilan tugaydigan barcha fayllarni ignore qiladi
- .env → parollar va API kalitli saqlangan faylni GitHubga chiqarmaydi

GitHubga push qilish bosqichlari

Undan oldin bitta repositorg ochamiz yangi:

1. git status
2. git add .
3. git commit -m "Project folderlar qo'shildi"
4. git remote add origin url (repositorymiz)
5. git branch -M main
6. git push -u origin main

No.

Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads

⋮

ohirida GitHubga o'tib bitta reload qilsak hammasini birga bir kochganini koramiz.

Agar biron yangi faylni qoshib keyin uni push qiladigan bolsak

① git add.

⇓

② git commit -m "New file added"

⇓

③ git push

↳ oxirida bitta reload

Bizga .gitkeep kerak sababi

Bu bosh papkani Git'da saqlab qolish uchun ishlatiladigan bosh fayl.

No.

src / preprocessing.py

```
Import pandas as pd
from sklearn.preprocessing import OneHotEncoder, StandardScaler

class DataPreprocessor:

    def __init__(self):
        self.onehot_cols = []
        self.label_cols = []
        self.num_cols = []

        self.encoder = OneHotEncoder(
            sparse_output = False,
            handle_unknown = "ignore"
        )
        self.scaler = StandardScaler()

# 1. Columnlarni aniqlash
    def find_columns(self, X_train):
        self.num_cols = X_train.select_dtypes(
            include = ["int64", "float64"]
        ).columns.tolist()

        cat_cols = X_train.select_dtypes(
            include = ["object", "category"]
        ).columns.tolist()
```

```
for col in cat_cols:
    if X_train[col].nunique() <= 5:
        self.onehot_cols.append(col)
    else:
        self.label_cols.append(col)

# 2. Encoding

def encode(self, X_train, X_test):
    X_train_enc = X_train[self.num_cols].copy()
    X_test_enc = X_test[self.num_cols].copy()

    # Label Encoding
    for col in self.label_cols:

        X_train_enc[col] = X_train[col].astype("category").
                                                cat.codes

        categories = (
            X_train[col]
            .astype("category")
            .cat.categories
        )

        X_test_enc[col] = pd.Categorical(
            X_test[col],
            categories = categories
        ).codes

    # One-Hot Encoding
```

No.

```
if len(self.onehot_cols)>0:

    train_ohe = self.encoder.fit_transform(
        X_train[self.onehot_cols]
    )
    test_ohe = self.encoder.transform(
        X_test[self.onehot_cols]
    )

    train_ohe = pd.DataFrame(
        train_ohe,
        columns = self.encoder.get_feature_
                  names_out(self.onehot_cols),
        index = X_train.index
    )

    test_ohe = pd.DataFrame(
        test_ohe,
        columns = self.encoder.get_feature_
                  names_out(self.onehot_cols),
        index = X_test.index
    )
    X_train_enc = pd.concat(
        [X_train_enc, train_ohe],
        axis = 1
    )
    X_test_enc = pd.concat(
        [X_test_enc, test_ohe],
        axis = 1
    )
return X_train_enc, X_test_enc
```

3. Scaling

```
def scale(self, X_train_enc, X_test_enc):
    X_train_scaled = X_train_enc.copy()
    X_test_scaled = X_test_enc.copy()

    X_train_scaled[self.num_cols] = self.scaler.fit_transform(
        X_train_scaled[self.num_cols]
    )

    X_test_scaled[self.num_cols] = self.scaler.transform(
        X_test_scaled[self.num_cols]
    )
    return X_train_scaled, X_test_scaled
```

Notebook (experiment.ipynb)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from src.preprocessing import DataPreprocessor
```

Dataset → Split

```
preprocessor = DataPreprocessor()
preprocessor.find_columns(X_train)

X_train_enc, X_test_enc = preprocessor.encode(
    X_train,
    X_test
)
```

No.

```
X_train_scaled, X_test_scaled = preprocessor.scale(
    X_train_enc,
    X_test_enc
)
```